FATEC POMPEIA

Introdução à Computação em Nuvem · ISW055 · 161_SIST. INTELIGENTES_N

P2

Relatório consolidado

arquitetura e atividades do 2º bimestre

Avaliação individual · 2026.2

Marcio Gabriel Schinor Mazega

Produto: **Nivelo**
https://github.com/Schinor/Nivelo

Prof. Allan Lincoln Rodrigues Siriani
Pompeia, 13/11/2026

Sumário

1 Introdução 2
2 Metodologia 2
3 Arquitetura do SaaS 2
3.1 Visão geral e componentes 2
3.2 Onde roda 4
3.3 Fluxo de uma requisição 4
3.4 Fluxo de assinatura 4
3.5 Modelo de dados do domínio 5
3.6 Segurança 6
3.7 Observabilidade, CI/CD e documentação da API 7
3.8 Camadas IaaS, PaaS e SaaS do projeto 7
4 Quadro de entregas 7
5 Atividades realizadas 8
5.1 Atividade 7 — Plano Premium — cobrança de verdade com Stripe 8
5.2 Atividade 8 — SaaS, PaaS, IaaS — e a documentação que o projeto nunca teve 11
5.3 Atividade 9 — Fork e plano de negócio 13
5.4 Atividade 10 — Seu negócio no ar — o domínio no lugar dos filmes 16
5.5 Atividade 11 — SaaS completo — planos, limites e assinatura com Stripe 18
6 Considerações finais 21
7 Declaração de autoria 21

1 Introdução

A disciplina Introdução à Computação em Nuvem propõe construir, ao longo do semestre, um sistema real que rode numa infraestrutura de nuvem: containers, microsserviços, autenticação com papéis, auditoria, armazenamento de objetos, observabilidade, entrega contínua e, no 2º bimestre, cobrança recorrente. No 1º bimestre, esse sistema foi o HANK+, um catálogo de títulos com favoritos, comentários e perfis, publicado numa stack do Portainer com subdomínio próprio.

No 2º bimestre, o HANK+ virou o **Nivelo**, um SaaS de diário de obra (RDO) para pequenos empreiteiros. A dor é concreta: o empreiteiro que toca de 2 a 10 reformas ao mesmo tempo não consegue provar ao cliente o que foi feito, quando e por que a obra atrasou. A última parcela do contrato fica retida enquanto se discute, e o registro está espalhado em fotos de WhatsApp e cadernos de canteiro. O Nivelo transforma as fotos que o empreiteiro já tira num diário com data, clima, ocorrências e histórico imutável, que o cliente acompanha pelo próprio celular. A cobrança é por empresa: o Free cobre uma obra ativa, e o Pro (R$ 29,90/mês) e o Construtora (R$ 69,90/mês) liberam obras e equipe.

Este relatório descreve a arquitetura do sistema que está no ar e apresenta, uma a uma, as atividades 7 a 11, com datas, evidências e dificuldades. As seções seguem esta ordem: metodologia, arquitetura do SaaS, quadro de entregas, fichas das atividades e considerações finais.

2 Metodologia

As atividades 7 e 8 foram feitas no repositório público do HANK+ (`Schinor/API-Filmes-Allan`), o mesmo do 1º bimestre. Na atividade 9, esse repositório foi espelhado com todo o histórico para um repositório novo, `Schinor/Nivelo`, onde o SaaS foi desenvolvido nas atividades 10 e 11, na mesma stack do Portainer e no mesmo subdomínio. Por isso, os commits das atividades 7 e 8 aparecem nos dois repositórios com o mesmo identificador.

O desenvolvimento seguiu um roteiro por fases, com um commit por tarefa em Conventional Commits e decisões registradas em ADRs (`docs/adr/`). Cada mudança passa pelo pipeline do GitHub Actions (lint, tipos, fronteiras de camada, testes e subida da stack completa) antes de virar imagem no GHCR. A data e a hora de cada entrega foram extraídas do histórico de commits (`git log`, horário de Brasília), conferidas na página do commit no GitHub e, quando existe, na issue de entrega que menciona o professor.

As telas do sistema foram capturadas em produção por um roteiro automatizado com Playwright (`e2e/navegar.py`), que navega como um celular de 390 px, cria uma conta nova, assina o plano com o cartão de teste do Stripe e tira um print de cada passo. As páginas do GitHub foram capturadas da mesma forma.

3 Arquitetura do SaaS

3.1 Visão geral e componentes

O sistema é uma stack de sete containers no Portainer, ligados por uma rede Docker interna (`nivelo-network`). Só o serviço `api` publica porta; o domínio público chega até ela por um túnel da Cloudflare. Os demais serviços só existem na rede interna, e o banco MariaDB é compartilhado e fica fora da stack.

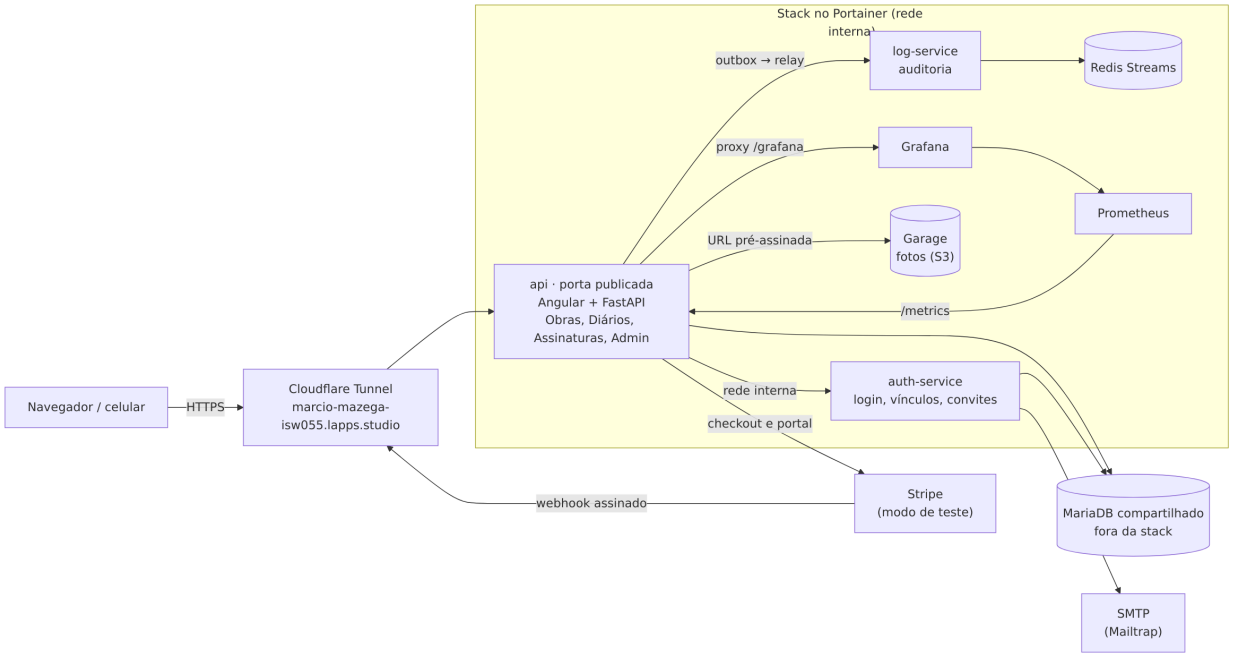


Figura 1: Diagrama de componentes do Nivelo em produção

Componente	Responsabilidade	Tecnologia / imagem	Público?
api	telas (Angular), API de obras, diários, fotos, assinaturas e painel admin, webhook do Stripe, proxy do Grafana	FastAPI + Angular · `ghcr.io/schinor/nivelo-api`	sim (domínio)
auth-service	login, cadastro da empresa, vínculos e papéis, convites, recuperação de senha	FastAPI · `nivelo-auth-service`	não
log-service	recebe os eventos de auditoria da outbox e os expõe ao painel	FastAPI · `nivelo-log-service`	não
Redis	armazena os eventos de auditoria (Redis Streams)	`redis:7-alpine`	não
MariaDB	dados do domínio, usuários, vínculos, assinaturas, eventos do Stripe	MariaDB compartilhado	não
Garage	fotos dos diários e de perfil, compatível com S3, por URL pré-assinada	`nivelo-garage`	não
Prometheus e Grafana	métricas técnicas e de negócio (MRR, assinaturas ativas)	`nivelo-prometheus`, `nivelo-grafana`	por proxy, com senha
Stripe (externo)	checkout, assinaturas, portal do cliente, webhooks	serviço SaaS, modo de teste	webhook ⟶ api

O armazenamento de objetos é o **Garage**, e não o MinIO: ele já estava em uso desde a atividade de upload do 1º bimestre e cumpre o mesmo papel, com a API S3.

Dentro da `api`, o código segue a arquitetura hexagonal, com contextos delimitados (Gestão de Obras, Assinaturas, Administração). A camada `domain/` de cada contexto não importa FastAPI, SQLAlchemy nem o SDK do Stripe, e o `import-linter` barra no CI qualquer violação. O Stripe entra por uma camada

anticorrupção (`stripe_gateway.py`): o resto do código fala em `Plano` e `StatusAssinatura`, nunca em objetos do SDK.

3.2 Onde roda

A stack roda no Portainer da VM da Fatec (Proxmox), a mesma do 1º bimestre, com a URL pública `https://marcio-mazega-isw055.lapps.studio`. O Portainer não faz build: ele lê o `docker-compose.yml` do repositório `Schinor/Nivelo` e puxa as imagens `ghcr.io/schinor/nivelo-*` publicadas pelo CI, fixadas pela variável `IMAGE_TAG` (`sha-<commit>`). As migrações do banco rodam sozinhas no boot da imagem (`alembic upgrade head`).

Os sete containers (`api`, `auth-service`, `log-service`, `redis`, `garage`, `prometheus`, `grafana`) têm limite de 1 GB de memória cada, e apenas a `api` publica porta. As variáveis de ambiente ficam na stack do Portainer e só os nomes estão no `.env.example`: `DATABASE_URL`, `SECRET_KEY`, `AUTH_INTERNAL_TOKEN`, `LOG_INTERNAL_TOKEN`, `STORAGE_ACCESS_KEY`, `STORAGE_SECRET_KEY`, `GARAGE_RPC_SECRET`, `STRIPE_SECRET_KEY`, `STRIPE_WEBHOOK_SECRET`, `STRIPE_PRICE_PRO`, `STRIPE_PRICE_CONSTRUTORA`, `FRONTEND_URL`, `APP_BASE_URL`, `ADMIN_EMAIL`, `ADMIN_SENHA_INICIAL`, `MAILTRAP_*`, `GRAFANA_ADMIN_PASSWORD`, `PROMETHEUS_PROXY_PASSWORD` e `IMAGE_TAG`.

3.3 Fluxo de uma requisição

Exemplo: o encarregado publica o diário do dia com uma foto.

1. O celular envia `POST /api/obras/{id}/diarios` com o token JWT ao subdomínio; a Cloudflare encaminha para a porta da `api`.
2. A `api` valida o token localmente (Padrão B: `sub`, `empresa_id`, `papel` e `admin` vêm no próprio token, assinado com a `SECRET_KEY` compartilhada com o auth-service). A empresa vem **só** do token, nunca do corpo ou da URL.
3. A rota confere o papel: o encarregado só registra diário nas obras de que é membro; um cliente recebe **403**. Uma obra de outra empresa responde **404**, sem revelar que existe.
4. O caso de uso valida as regras do domínio (data não futura, atividades preenchidas para publicar, um diário por obra e dia) e grava o diário no MariaDB. Na mesma transação, grava o evento `diario.publicado` na tabela `outbox`.
5. A foto vai direto do navegador para o Garage por uma URL pré-assinada emitida pela `api`; a confirmação grava `diario_fotos` e o evento `foto.anexada`.
6. Um relay lê a outbox e entrega os eventos ao log-service, que os guarda no Redis Streams. O painel do admin e o histórico da obra mostram quem fez o quê e quando.
7. A `api` responde 201 com o diário; a tela mostra o selo “Publicado”.

3.4 Fluxo de assinatura

A assinatura pertence à **empresa**, e só o dono assina. A volta do checkout não muda nada: quem muda o plano é exclusivamente o webhook, e o plano é consultado no banco a cada requisição (nunca vai no JWT).

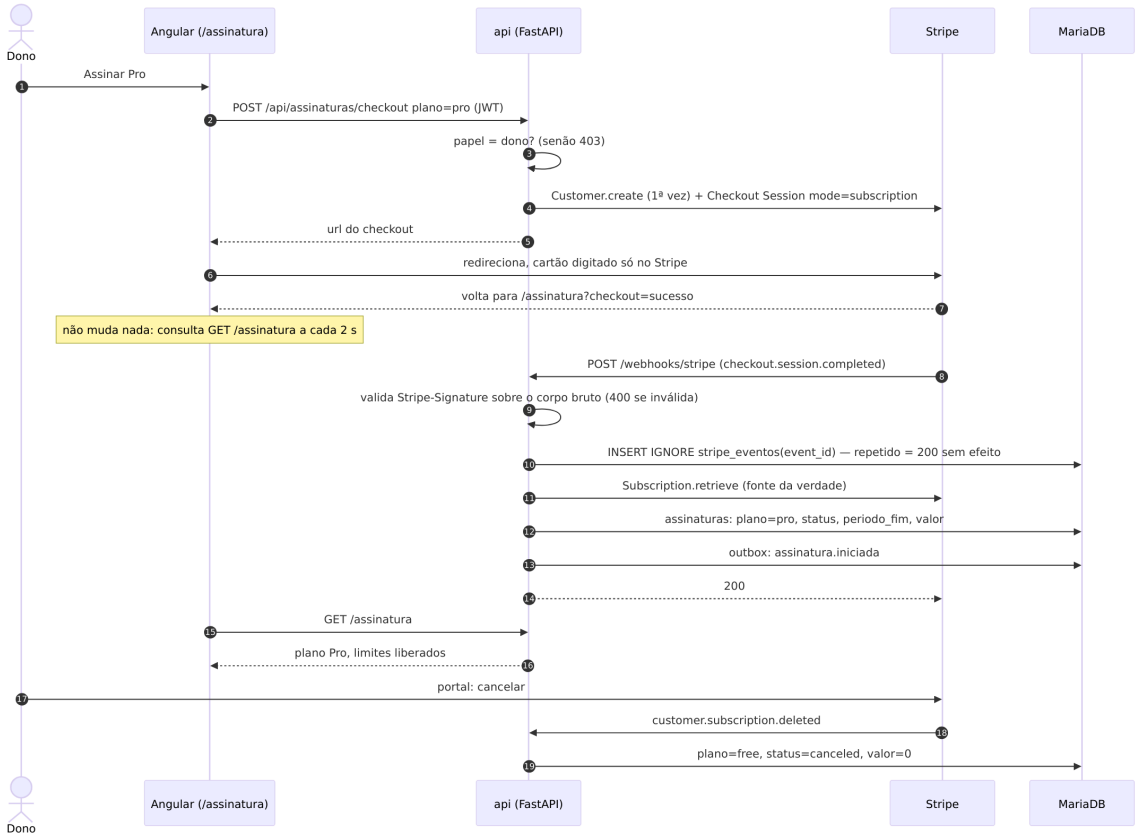


Figura 2: Fluxo de assinatura: checkout ⟶ webhook ⟶ plano no banco ⟶ limites

1. **Checkout.** `POST /api/assinaturas/checkout` (só o dono; outro papel recebe 403) cria ou reutiliza o customer do Stripe com `metadata.empresa_id` e abre uma Checkout Session com `mode=subscription`, `client_reference_id` igual à empresa e o `price_id` do plano. O cartão é digitado só na página hospedada do Stripe, o que mantém o sistema fora do escopo do PCI-DSS.
2. **Webhook.** `POST /webhooks/stripe` é público e sem JWT. A assinatura `Stripe-Signature` é validada sobre o **corpo bruto**; se falhar, responde 400 e registra `webhook.assinatura_invalida`.
3. **Idempotência.** O `event_id` entra em `stripe_eventos` com `INSERT IGNORE`; um evento já processado responde 200 sem efeito. Além disso, para qualquer um dos quatro eventos (`checkout.session.completed`, `customer.subscription.updated`, `customer.subscription.deleted`, `invoice.payment_failed`), a `api` busca a assinatura atual no Stripe e grava esse estado. Processar duas vezes, ou fora de ordem, grava o mesmo resultado.
4. **Limites.** Os direitos de cada plano (obras ativas, fotos por obra, pessoas na equipe) são calculados a partir do plano **e** do status. Uma assinatura `past_due` volta aos limites do Free sem apagar nada. A checagem acontece com `SELECT ... FOR UPDATE` na transação do comando, para duas requisições simultâneas não criarem duas obras no Free. Ao estourar o limite, a API responde **402** com um Problem Details de tipo estável.
5. **Portal e cancelamento.** `POST /api/assinaturas/portal` abre o Billing Portal do Stripe. O cancelamento dispara `customer.subscription.deleted`, e a empresa volta para o Free com valor zero.

3.5 Modelo de dados do domínio

Toda tabela de domínio tem `empresa_id`. Um evento do SQLAlchemy aplica o filtro da empresa ativa a toda consulta ORM, e uma consulta sem empresa definida falha em vez de devolver tudo. O usuário é global; o papel de negócio mora em `vinculos`, e a assinatura liga a empresa ao Stripe.

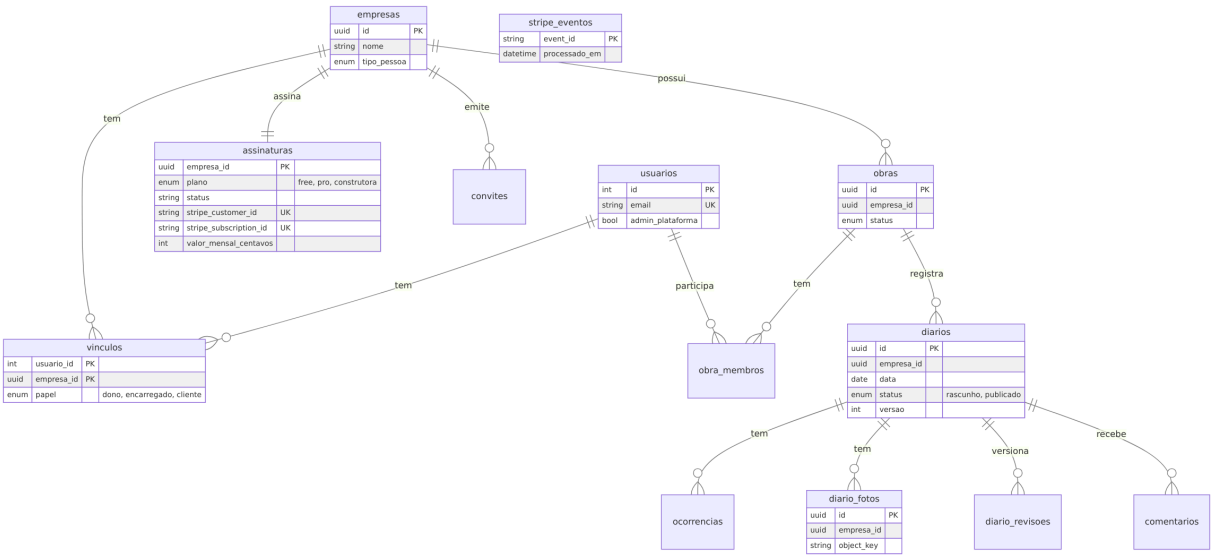


Figura 3: Modelo de dados do domínio (tabelas principais)

As tabelas de Identidade (`usuarios`, `empresas`, `vinculos`, `convites`, `reset_tokens`) são do auth-service. As de Gestão de Obras (`obras`, `obra_membros`, `diarios`, `ocorrencias`, `diario_fotos`, `diario_revisoes`, `comentarios`) e de Assinaturas (`assinaturas`, `stripe_eventos`) são da `api`, com a `outbox` compartilhada para os eventos. Cada serviço versiona as próprias migrações no Alembic, e o SQL exportado fica em `db/`.

3.6 Segurança

Os papéis vêm do vínculo do usuário com cada empresa: a mesma pessoa pode ser encarregada numa construtora e cliente de outro empreiteiro. O `admin` é da plataforma, criado por variável de ambiente no primeiro boot, e não lê dados de obras.

Ação	admin	dono	encarregado	cliente
Ver o painel admin e o log global	sim	não	não	não
Criar, editar, remover obra	não	sim	não	não
Convidar pessoas e gerir membros	não	sim	não	não
Ver obras	não	todas da empresa	as que é membro	as que é membro
Registrar e publicar diário	não	sim	nas suas obras	não
Anexar ou remover foto	não	sim	nas suas obras	não
Comentar em diário publicado	não	sim	sim	sim
Assinar ou gerenciar o plano	não	sim	não	não
Ver dados de outra empresa	não	não (404)	não (404)	não (404)

Quem decide é a API. Esconder o botão é só conforto: ação proibida para o papel responde 403 e gera o evento `acesso.negado`. O isolamento entre empresas tem duas camadas: a empresa vem só do token,

e o filtro automático do ORM faz qualquer consulta sem empresa falhar. Os testes de API cobrem, por recurso, pelo menos um caso de 404 entre empresas e um de 403 por papel, também no MariaDB.

Segredos. Nenhuma chave está no repositório: os valores ficam nas variáveis da stack, e o `.env.example` tem só os nomes. A busca `git grep -nE "sk_(test|live)_|whsec_"` volta vazia. A API recusa subir com uma chave de produção do Stripe. O auth-service não tem porta publicada, e as rotas internas entre serviços exigem um token compartilhado. O cadastro público ignora qualquer papel enviado no corpo e cria sempre o dono de uma empresa nova.

3.7 Observabilidade, CI/CD e documentação da API

Observabilidade. Cada serviço expõe `/health/live`, `/health/ready` e `/metrics`. O Prometheus coleta as métricas, e o Grafana, servido em `/grafana` pelo proxy da `api`, tem painéis técnicos e de negócio, com os gauges `nivelo_mrr_centavos` e `nivelo_assinaturas_ativas`. Cada requisição carrega um `X-Request-ID`, que aparece nos logs e nos erros em Problem Details (RFC 9457).

CI/CD. O workflow `ci-cd.yml` do GitHub Actions roda, a cada push: ruff, mypy (estrito no domínio), import-linter, a busca pelo domínio antigo, os testes com pytest (541 passando), os testes de tenancy e de concorrência de novo no MariaDB 11.8, a verificação das migrações e a subida da stack completa com testes de fumaça (health, cadeia api → auth → banco, `/metrics` e a ponte do Garage). Com tudo verde, as imagens vão para o GHCR com a tag `sha-<commit>`.

Documentação da API. O Swagger fica em `/api/docs`, gerado do OpenAPI do FastAPI. O cliente TypeScript do front é gerado a partir dessa mesma especificação.

3.8 Camadas IaaS, PaaS e SaaS do projeto

Camada	O que é no Nivelo	Quem gerencia
IaaS	a VM no Proxmox da Fatec (Ubuntu + Docker), com rede e disco	o professor / Fatec
PaaS	o Portainer como plataforma de deploy (entregamos só o compose e as imagens); o MariaDB compartilhado, do qual recebemos só a `DATABASE_URL`; os nossos containers (`api`, `auth-service`, `log-service`) e os de terceiros hospedados por nós (Redis, Garage, Prometheus, Grafana)	Portainer e MariaDB: professor; containers: o aluno
SaaS	o próprio Nivelo, para o empreiteiro e o cliente da obra; e os SaaS que ele consome: Stripe (pagamento), Mailtrap (e-mail), Cloudflare (domínio e TLS), GitHub Actions e GHCR (CI/CD e imagens)	os fornecedores; o Nivelo, pelo aluno

O Garage é um bom exemplo da fronteira: ele cumpre o papel do Amazon S3, mas, como roda na nossa stack, é PaaS operado por nós. Se usássemos o S3 da AWS, o armazenamento subiria para SaaS.

4 Quadro de entregas

A tabela é gerada a partir das fichas da seção seguinte. As datas são do commit de entrega (atividades 7 e 8) ou, da atividade 9 em diante, da issue de entrega; quando a issue não existe, vale o commit e isso está dito na ficha.

Nº	Atividade	Data planejada	Data realizada	Situação
7	Plano Premium — cobrança de verdade com Stripe	09/10/2026	11/10/2026 19:08	entregue com atraso
8	SaaS, PaaS, IaaS — e a documentação que o projeto nunca teve	16/10/2026	14/10/2026 09:48	entregue
9	Fork e plano de negócio	23/10/2026	02/11/2026 19:48	entregue com atraso
10	Seu negócio no ar — o domínio no lugar dos filmes	06/11/2026	09/11/2026 21:52	entregue com atraso
11	SaaS completo — planos, limites e assinatura com Stripe	13/11/2026	13/11/2026	entregue

5 Atividades realizadas

5.1 Atividade 7 — Plano Premium — cobrança de verdade com Stripe

Descrição	Plano pago em modo de teste: Checkout Session do Stripe e webhook marcando o usuário como premium.
Data planejada	09/10/2026
Data realizada	11/10/2026 19:08
Situação	entregue com atraso
Evidência	Commit 23c9f6b (README da atividade 7) e issue de entrega #10, aberta em 02/11/2026 18:06, com prints do checkout e do usuário premium
Link	https://github.com/Schinor/Nivelo/commit/23c9f6ba330c280902ba47f47d82bc0756cbbdd4

O que foi feito. O HANK+ ganhou um Plano Premium de R$ 9,90/mês, cobrado pelo Stripe em modo de teste. O benefício é verificado no backend: no plano gratuito o usuário guarda até 5 favoritos e, acima disso, recebe 403 com a mensagem do limite; o Premium não tem limite. O botão "Assinar" chama `POST /api/premium/checkout`, que cria uma Checkout Session com `mode=subscription` e `client_reference_id` igual ao usuário. O cartão é digitado só na página do Stripe.

A ativação acontece só pelo webhook assinado. Como o auth-service não tem porta publicada, o checkout e o webhook ficaram no catálogo, que já era exposto, e a gravação do status continuou com o dono da tabela `usuarios`: o catálogo valida a assinatura e repassa só os IDs ao auth-service pela rede interna, que grava `premium` e o `event_id` na mesma transação (idempotência). O plano nunca é lido do JWT, e sim do banco, para valer na hora sem relogar.

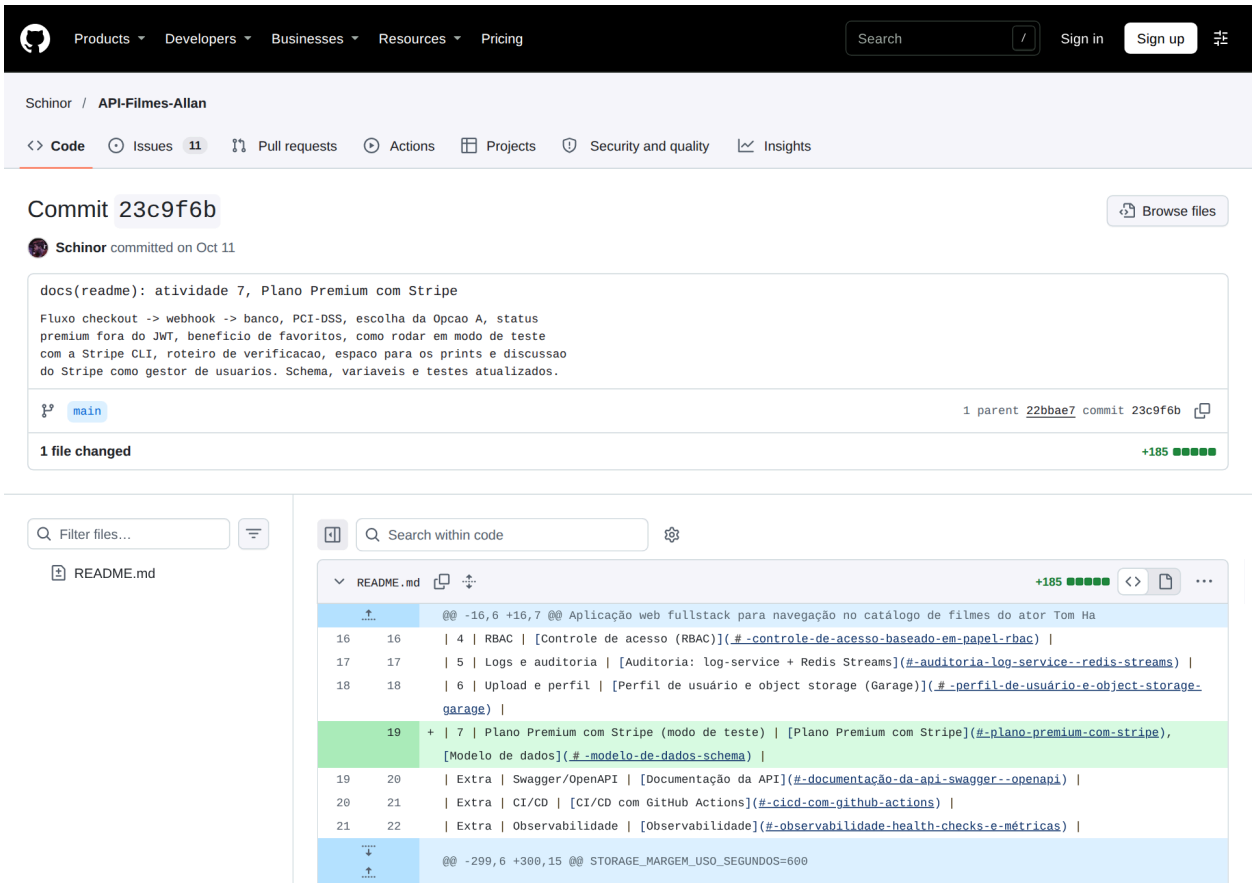


Figura 4: Atividade 7 — commit de entrega de 11/10/2026 (o repositório foi espelhado, então o mesmo commit está no Nivelo)

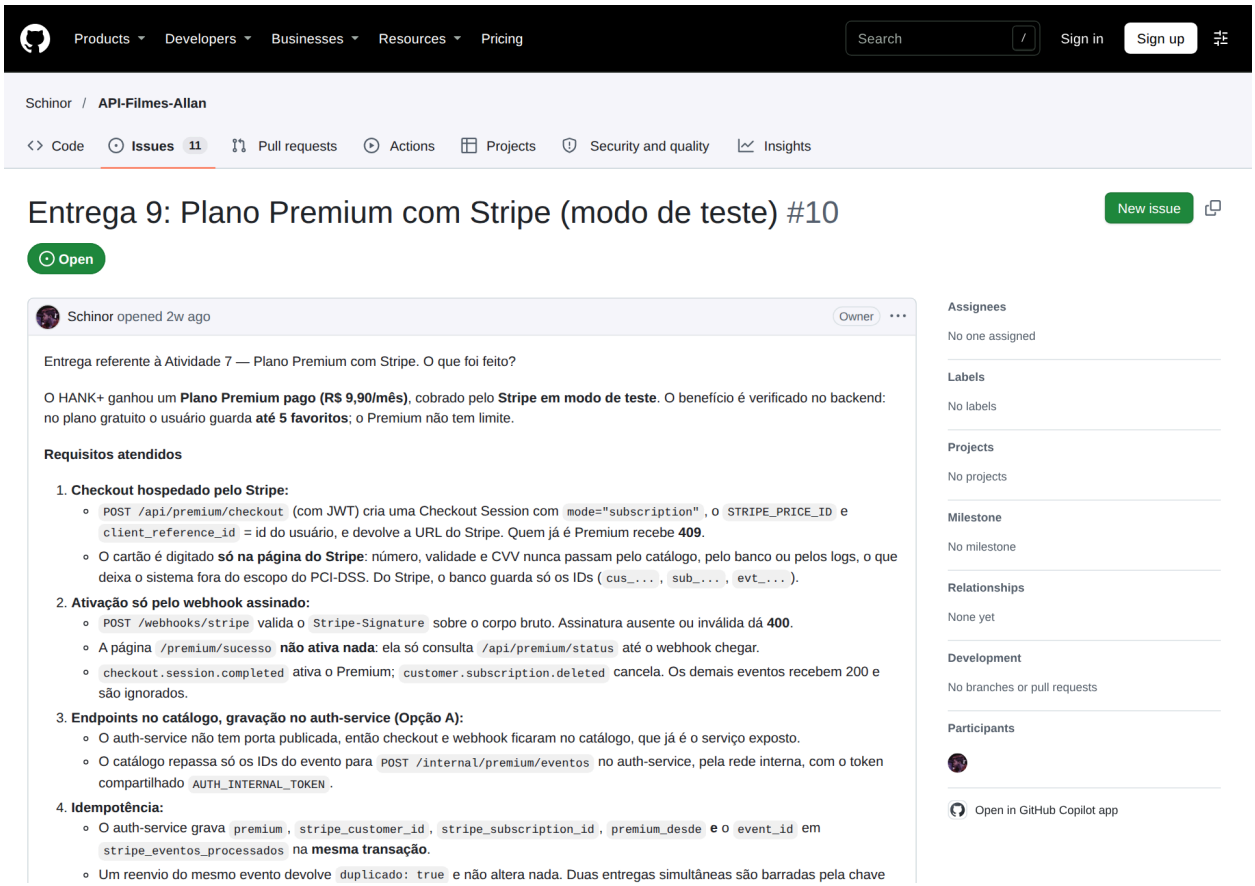


Figura 5: Atividade 7 — issue de entrega 10 no repositório do HANK+, mencionando o professor

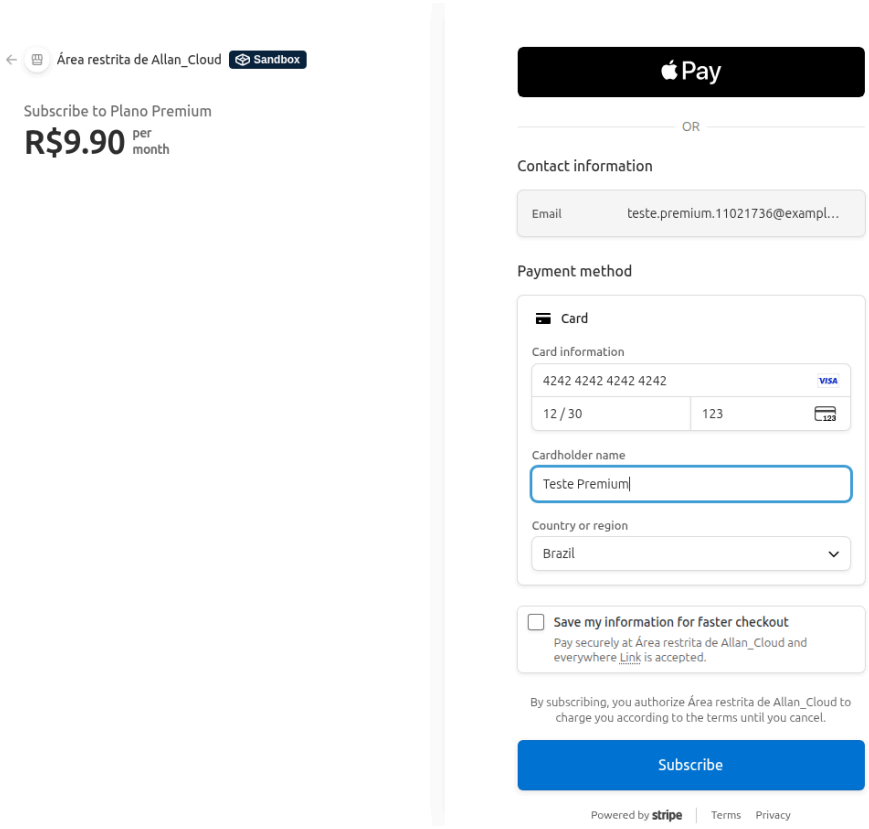


Figura 6: Atividade 7 — checkout hospedado do Stripe em modo de teste

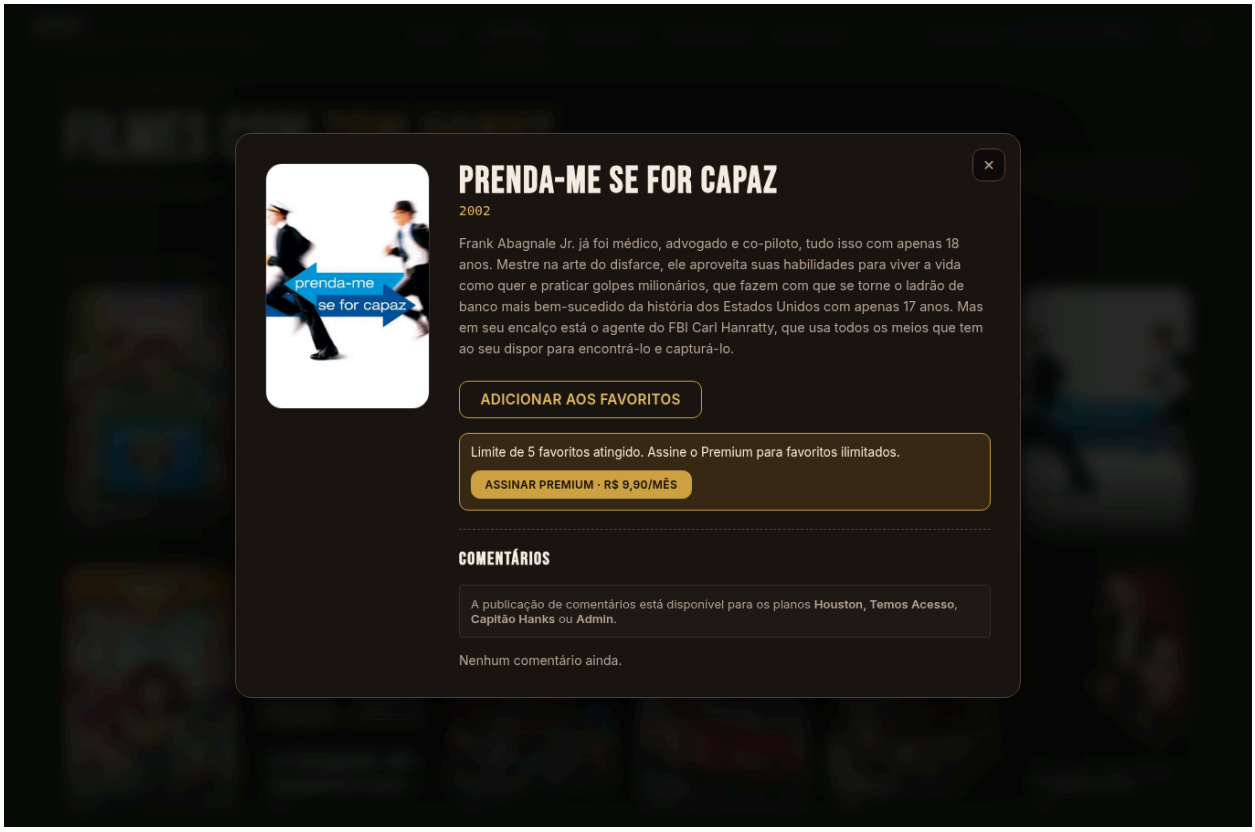


Figura 7: Atividade 7 — limite do plano gratuito bloqueando o 6º favorito

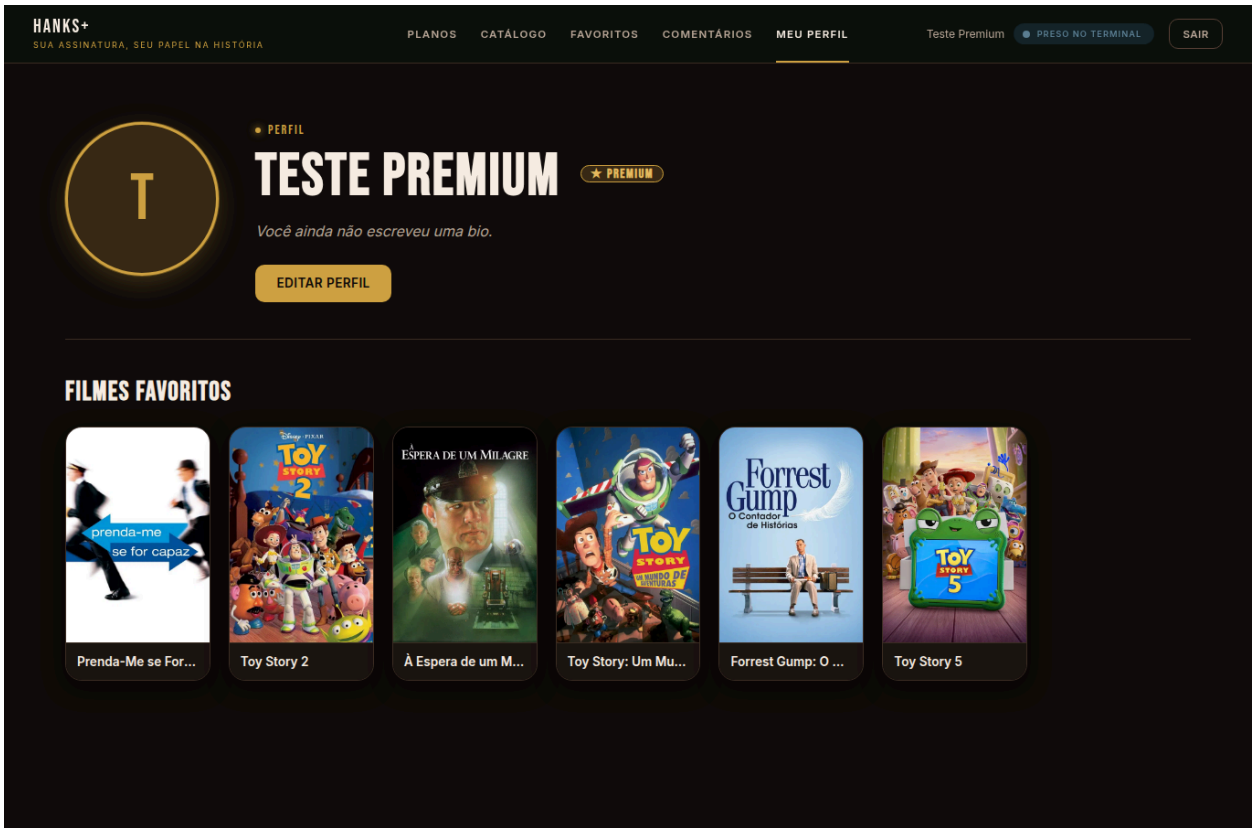


Figura 8: Atividade 7 — usuário Premium com os favoritos liberados

Dificuldades e como foram resolvidas. A página de sucesso cancelava a consulta de status antes de o webhook chegar e mostrava "não confirmado" mesmo com o pagamento aprovado; ela passou a consultar `/api/premium/status` a cada 1,5 s por até 15 s, sem alterar nada por conta própria. A escolha de onde pôr o webhook (catálogo ou auth-service) também pesou, porque o Stripe precisa alcançar a rota pela rede pública e o auth-service não tem porta. Os prints de produção só foram anexados em 02/11, junto com a issue de entrega.

5.2 Atividade 8 — SaaS, PaaS, IaaS — e a documentação que o projeto nunca teve

Descrição	Camadas de nuvem do próprio projeto e documentação reversa: especificação, arquitetura, regras de negócio e modelo de dados.
Data planejada	16/10/2026
Data realizada	14/10/2026 09:48
Situação	entregue
Evidência	Commit 3a2d56f (DOCUMENTACAO.md linkado no README) e issue de entrega #11, aberta em 02/11/2026 18:07
Link	https://github.com/Schinor/Nivelo/commit/3a2d56fdb7aa1a6683fb6c5d5991d4e35d4efe55

O que foi feito. O README ganhou a classificação de cada peça do HANK+ nas três camadas: a VM no Proxmox como IaaS (administrada pelo professor); o Portainer, os containers e o MariaDB compartilhado como PaaS; e o Stripe, o Mailtrap, a API externa de dados, a Cloudflare e o GitHub Actions como SaaS. Cada linha justifica a escolha, com o contraste entre o Garage auto-hospedado (PaaS) e o S3 da AWS (SaaS).

O `DOCUMENTACAO.md` (465 linhas) é a documentação reversa do sistema, escrita lendo o código: o que o sistema faz para quem usa, o diagrama de todos os serviços, 60 regras de negócio no formato "se X, então Y" e o modelo de dados.

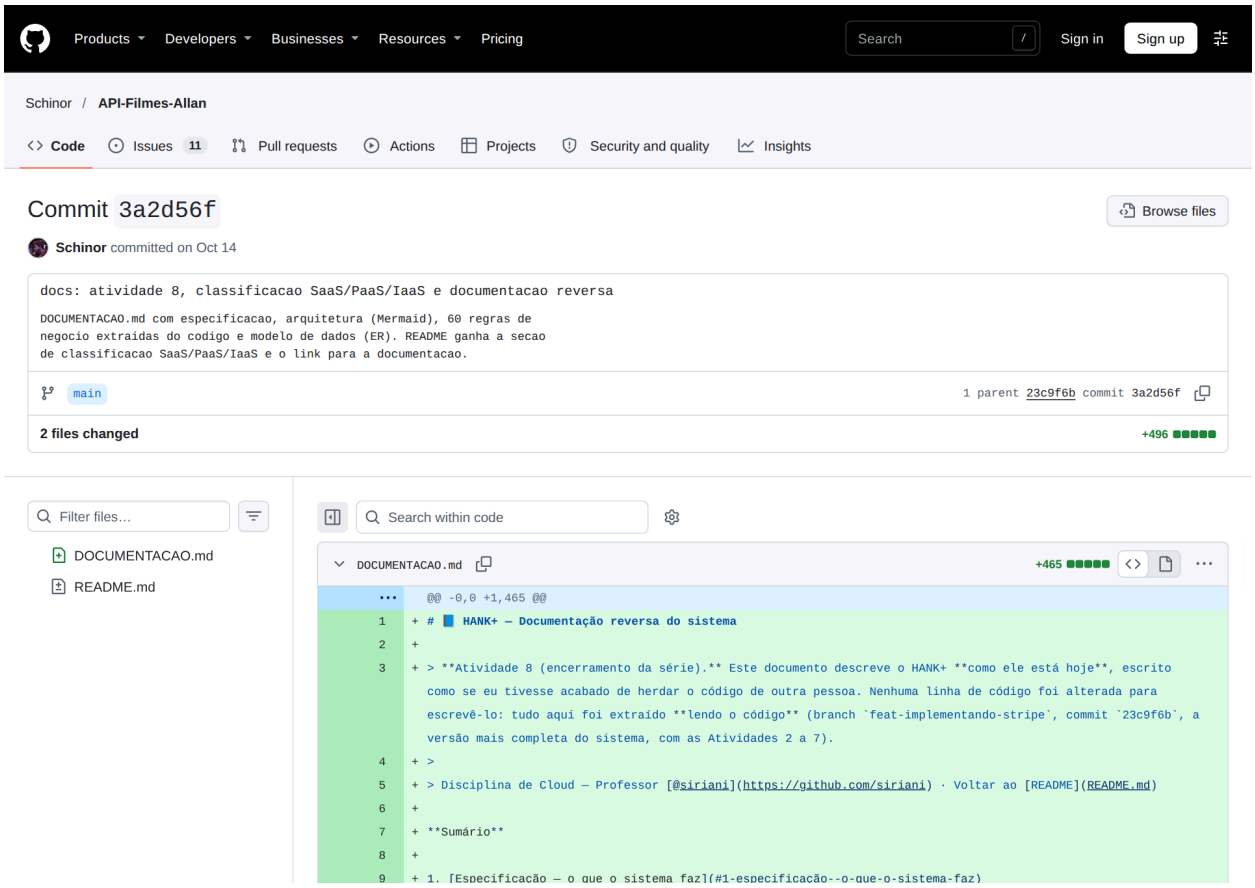


Figura 9: Atividade 8 — commit de entrega de 14/10/2026

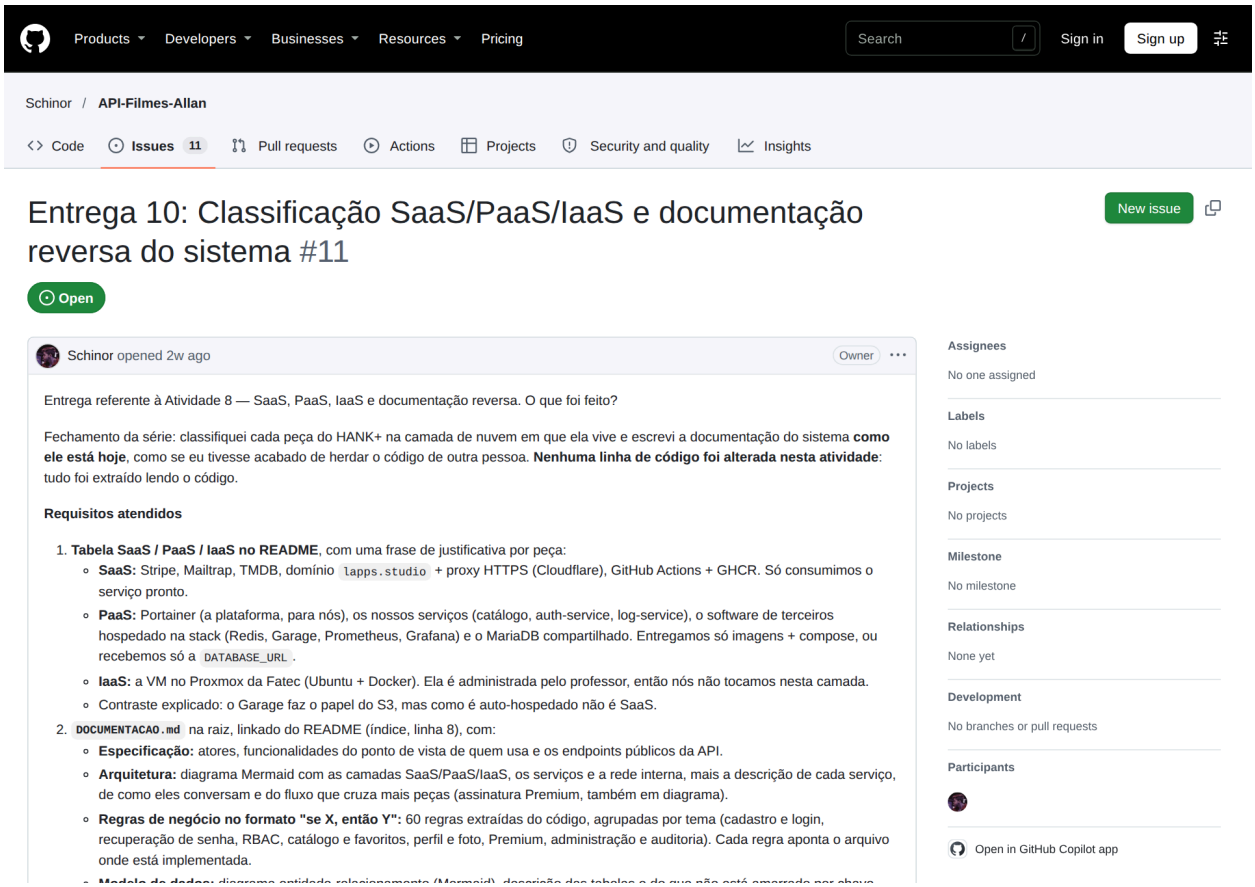


Figura 10: Atividade 8 — issue de entrega 11, mencionando o professor

Dificuldades e como foram resolvidas. A parte mais difícil foi a fronteira entre PaaS e SaaS em peças auto-hospedadas, como o Garage e o Grafana; o critério adotado foi "quem opera": se roda na nossa stack, é PaaS operado por nós. Na documentação reversa, o trabalho foi extrair as regras de negócio do código, e não da memória, o que revelou regras implícitas (por exemplo, o 409 ao favoritar duas vezes).

5.3 Atividade 9 — Fork e plano de negócio

Descrição	Repositório novo com o histórico do anterior, README do produto (dor, persona, Lean Canvas, modelo de receita, mapa de reaproveitamento) e backlog em issues.
Data planejada	23/10/2026
Data realizada	02/11/2026 19:48
Situação	entregue com atraso
Evidência	Repositório Schinor/Nivelo (criado em 02/11/2026), commit 9ff7a38 com o README do produto e as 12 issues do backlog. A issue de entrega (@siriani) só foi aberta em 13/11/2026
Link	https://github.com/Schinor/Nivelo

O que foi feito. O repositório do HANK+ foi espelhado com todo o histórico para `Schinor/Nivelo`. O README virou o plano de negócio: a dor do empreiteiro, com o custo em dinheiro, tempo e risco; a persona (Valdir, 47 anos, empreiteiro em Marília, 4 obras ao mesmo tempo); os concorrentes e alternativas; a proposta de valor; o modelo de receita por empresa (Free, Pro a R$ 29,90 e Construtora a R$ 69,90);

métricas de sucesso; o Lean Canvas; e o mapa de reaproveitamento, peça a peça, do HANK+ para o Nivelo. O backlog virou 12 issues, da modelagem das tabelas à limpeza do domínio antigo.

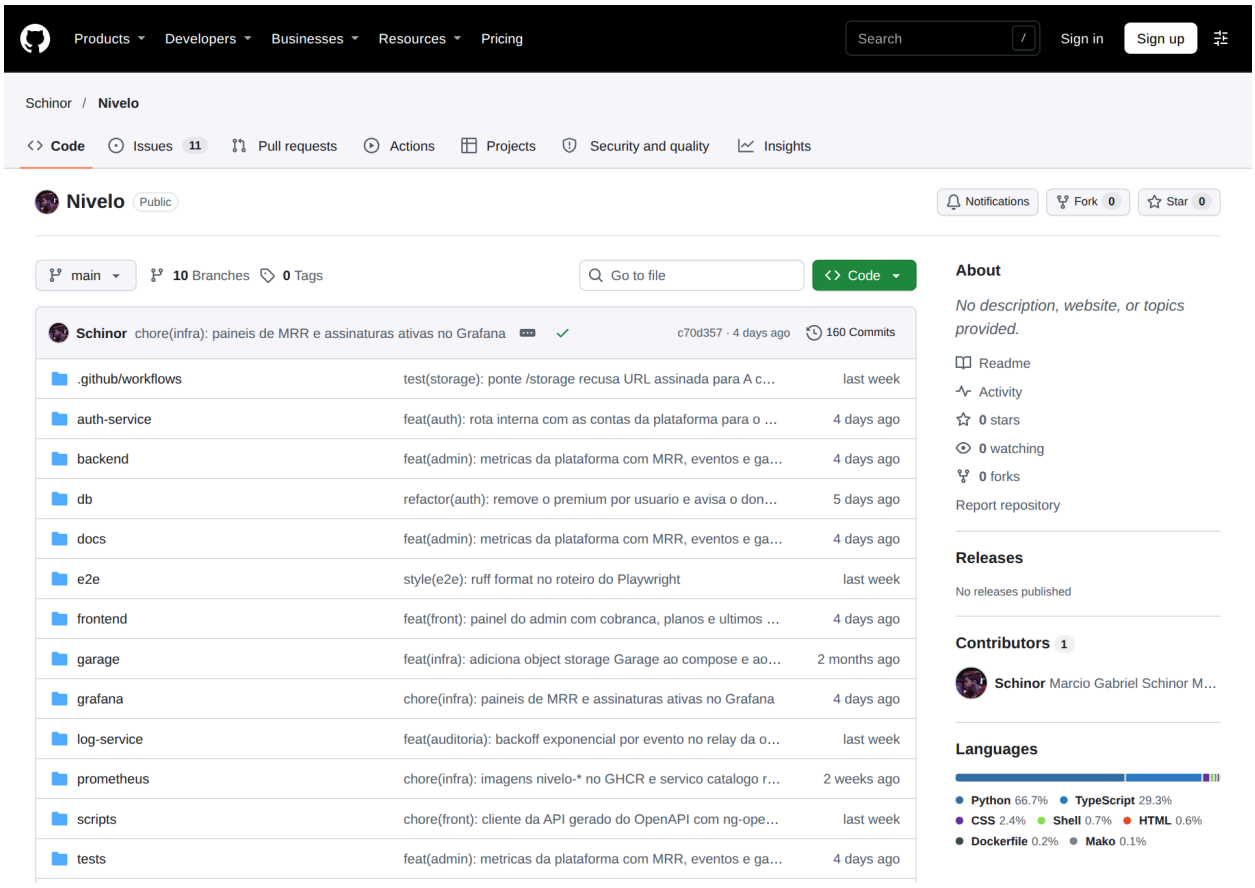


Figura 11: Atividade 9 — repositório novo com o README do produto

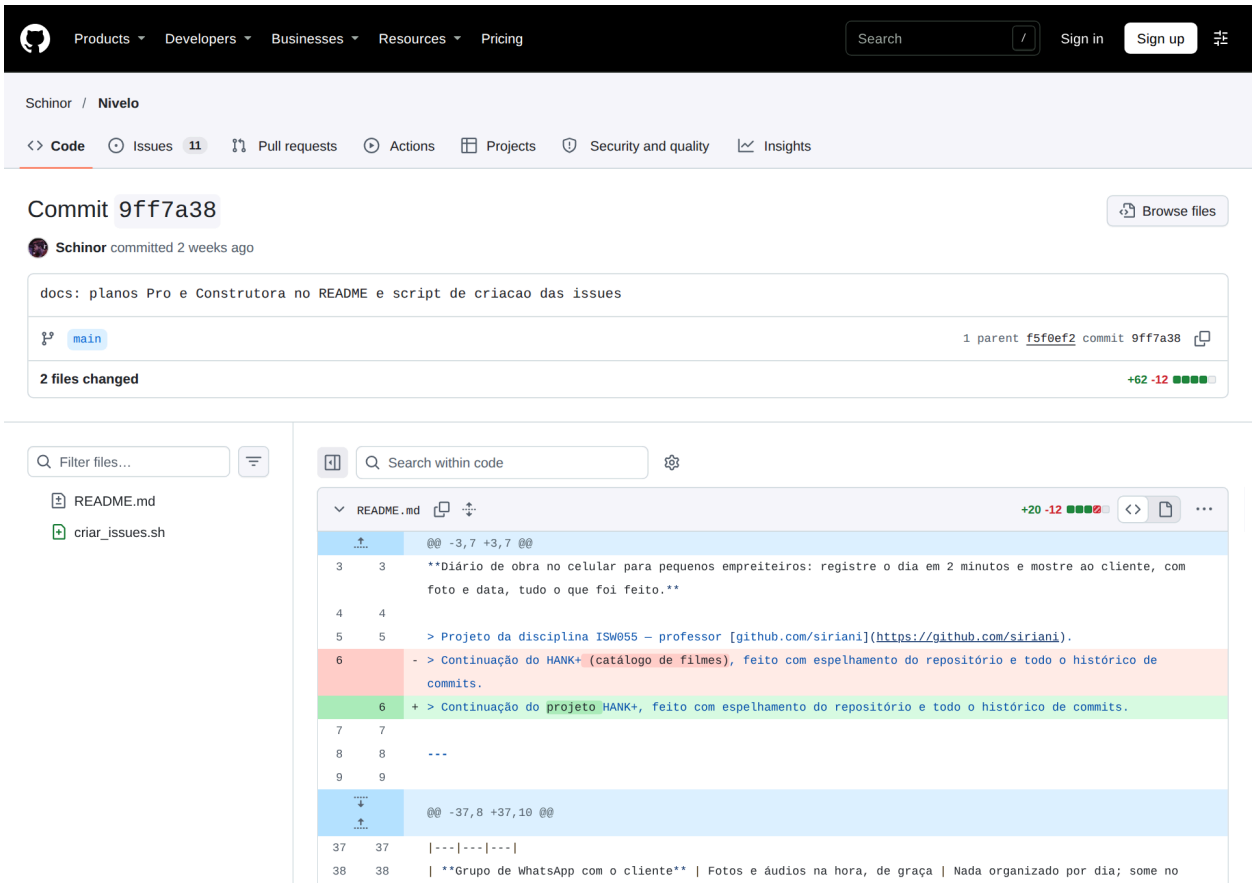


Figura 12: Atividade 9 — commit do plano de negócio de 02/11/2026

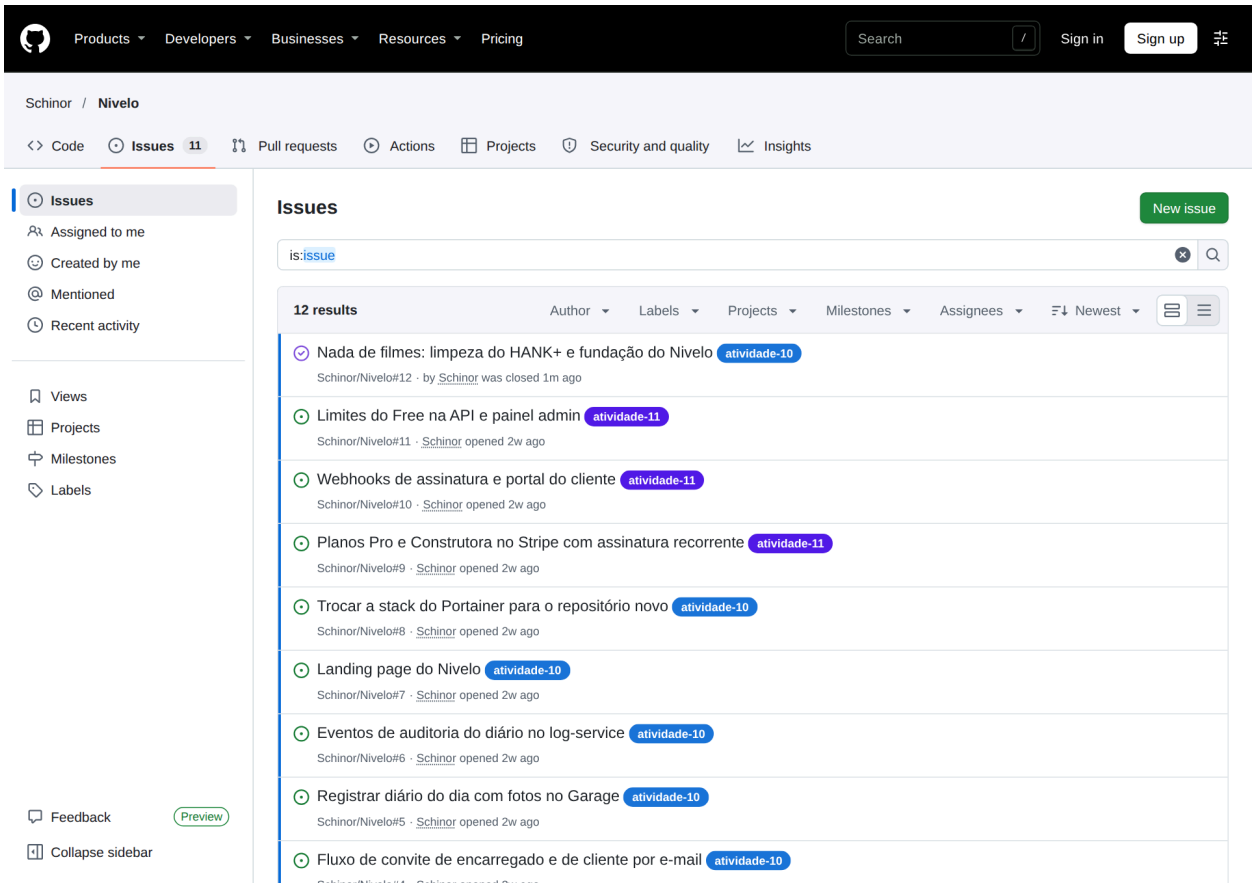


Figura 13: Atividade 9 — backlog em issues

Dificuldades e como foram resolvidas. A escolha do produto levou mais tempo que o previsto: foi preciso pesquisar concorrentes (Meu RDO, Prumo, Dia de Obra) e calibrar o limite do Free para que ele provasse valor numa obra inteira sem tornar o Pro dispensável. Isso atrasou a entrega para 02/11. A entrega foi registrada pelo commit e pelas issues do backlog; a issue de entrega própria, mencionando o professor, só foi aberta em 13/11/2026.

5.4 Atividade 10 — Seu negócio no ar — o domínio no lugar dos filmes

Descrição	Stack apontando para o repositório novo, tabelas e API do domínio, papéis do negócio, isolamento por usuário, eventos no log, upload e landing page.
Data planejada	06/11/2026
Data realizada	09/11/2026 21:52
Situação	entregue com atraso
Evidência	Commit 136fa3f (URL no ar, arquitetura no README e checklist em docs/ENTREGA_ATIVIDADE_10.md), enviado ao GitHub só em 13/11/2026, e o sistema no ar desde 09/11. Issue de entrega (@siriani) aberta em 13/11/2026
Link	https://github.com/Schinor/Nivelo/commit/136fa3f4726476fd5358f3f02969863f38dd36cb

O que foi feito. A stack do Portainer passou a puxar as imagens do repositório novo, no mesmo subdomínio. O domínio do Nivelo foi implementado em arquitetura hexagonal: empresas, vínculos e papéis (`dono`, `encarregado`, `cliente` e o `admin` da plataforma), obras, diários com clima, efetivo, ocorrências e revisões com motivo, fotos no Garage e comentários do cliente. Toda tabela tem `empresa_id`; a empresa vem só do token, um recurso de outra empresa responde 404 e ação proibida responde 403. Os eventos de negócio (`obra.criada`, `diario.publicado`, `foto.anexada`) saem por uma outbox para o log-service. A landing page apresenta o produto e os planos, e o front é mobile-first a 390 px.

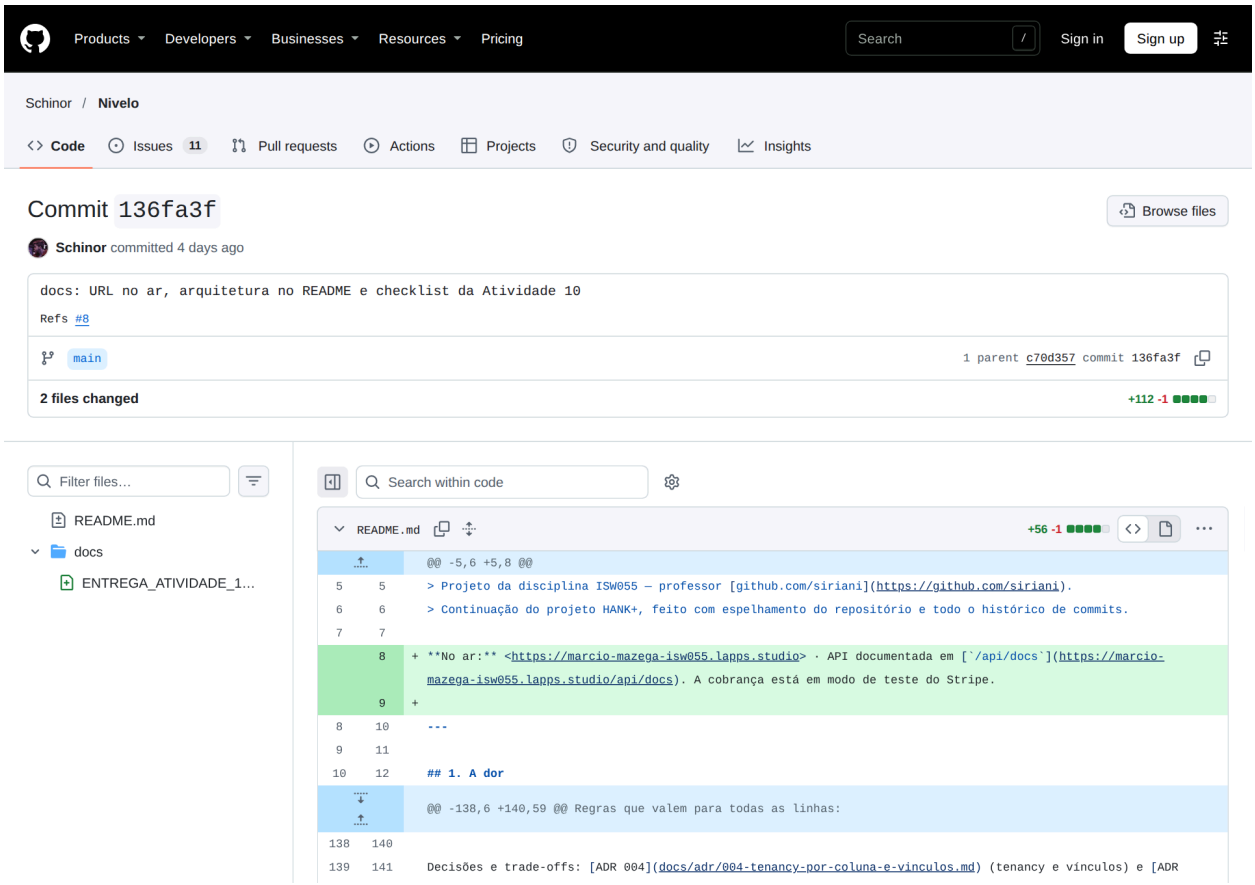


Figura 14: Atividade 10 — commit de entrega de 09/11/2026

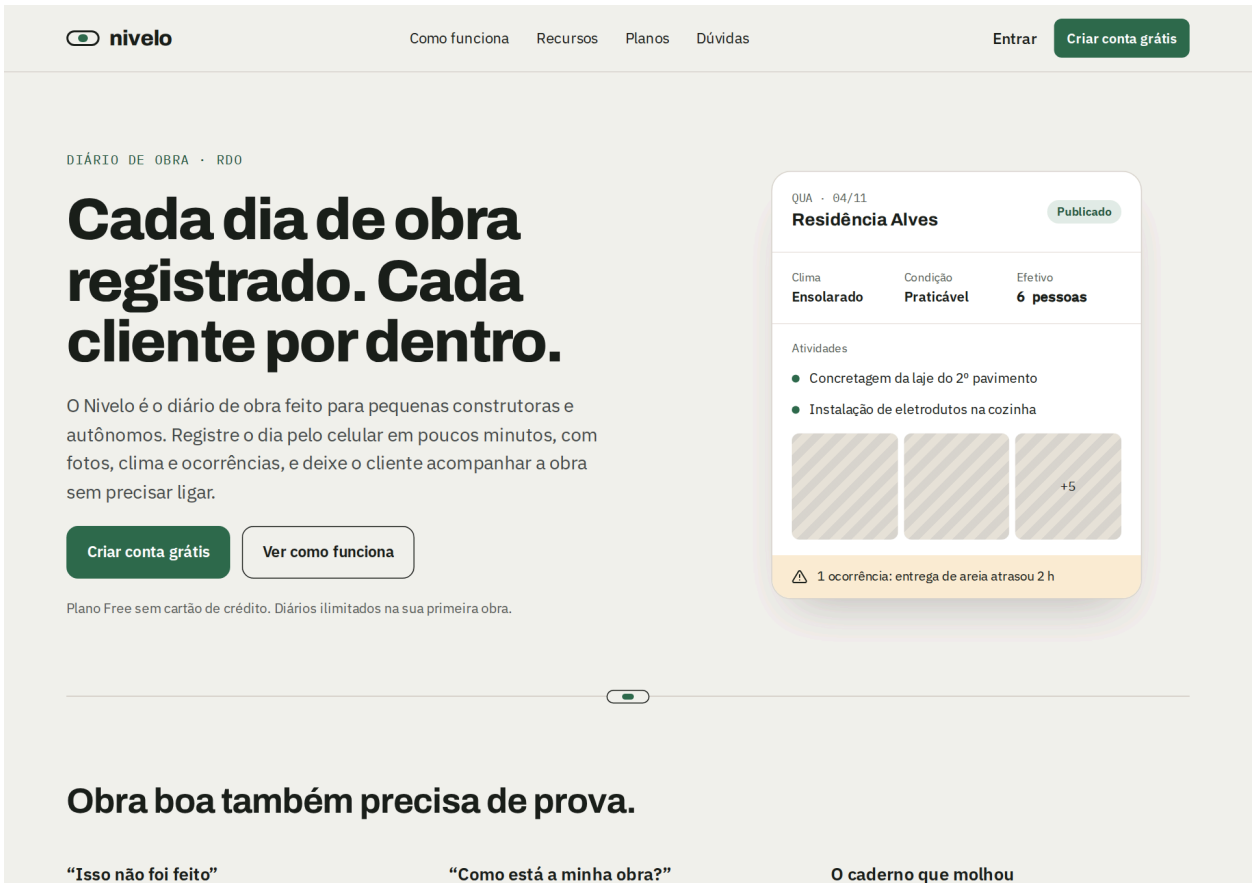


Figura 15: Atividade 10 — landing page em produção

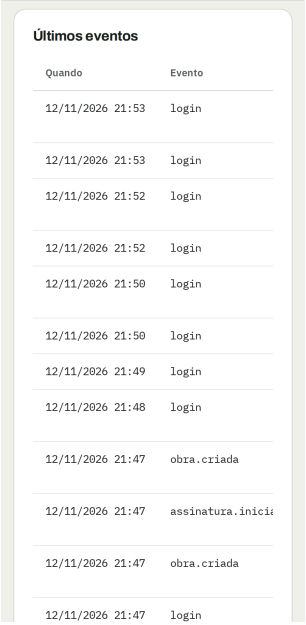


Figura 16: Atividade 10 — eventos de negócio no painel do admin, com quem e quando

Dificuldades e como foram resolvidas. A maior foi a multi-tenancy: o filtro por `empresa_id` não podia depender de cada desenvolvedor lembrar dele. A solução foi um evento do SQLAlchemy que aplica o filtro a toda consulta e faz falhar a consulta sem empresa definida. A segunda foi o fuso: o navegador em UTC mandava a data do dia seguinte depois das 21h, e o diário voltava 422 por "data futura"; o "hoje" passou a ser o do fuso do negócio. A troca da stack exigiu backup do banco e um caminho de volta documentado em `docs/DEPLOY.md`.

5.5 Atividade 11 — SaaS completo — planos, limites e assinatura com Stripe

Descrição	Planos recorrentes, checkout de assinatura, webhooks atualizando o plano, portal do cliente, limites por plano na API e painel admin.
Data planejada	13/11/2026
Data realizada	13/11/2026
Situação	entregue
Evidência	Issue de entrega (@siriani), commit, prints da assinatura com o cartão 4242, do Stripe em modo de teste, do webhook e do limite bloqueando
Link	https://github.com/Schinor/Nivelo

O que foi feito. O Stripe saiu do auth-service e virou o módulo `assinaturas` da `api` (Identidade não deve saber de cobrança), mantendo a mesma URL pública do webhook. A assinatura é por empresa: o checkout cria o customer com `metadata.empresa_id` e a Checkout Session em `mode=subscription`; o webhook valida a assinatura sobre o corpo bruto, registra o `event_id` (idempotência) e sincroniza a partir da assinatura atual no Stripe, o que torna o processamento imune a eventos repetidos ou fora de ordem. Os limites do Free (1 obra ativa, 30 fotos por obra, 1 encarregado) são aplicados na API com trava no banco e respondem 402. O painel do admin mostra empresas e usuários por plano, assinaturas ativas, inadimplentes, o MRR de teste e os últimos eventos.

Todos os prints abaixo foram tirados em produção em 12/11/2026, por um roteiro automatizado, com uma conta criada na hora.

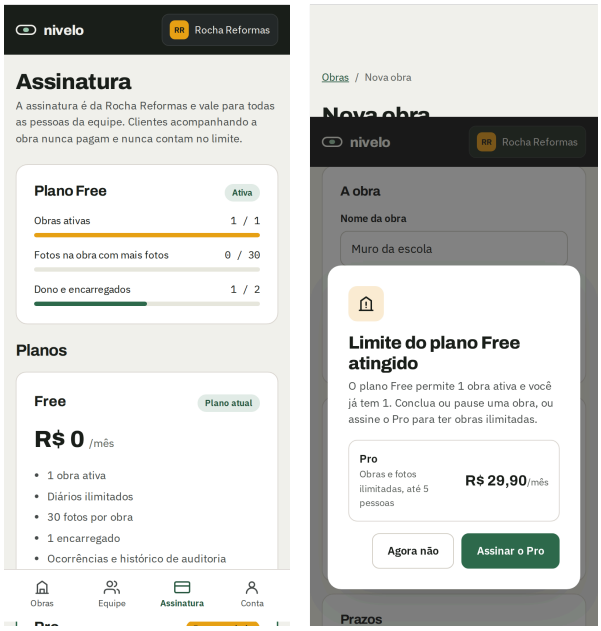


Figura 17: Atividade 11 — conta nova no Free e a 2ª obra recusada com 402

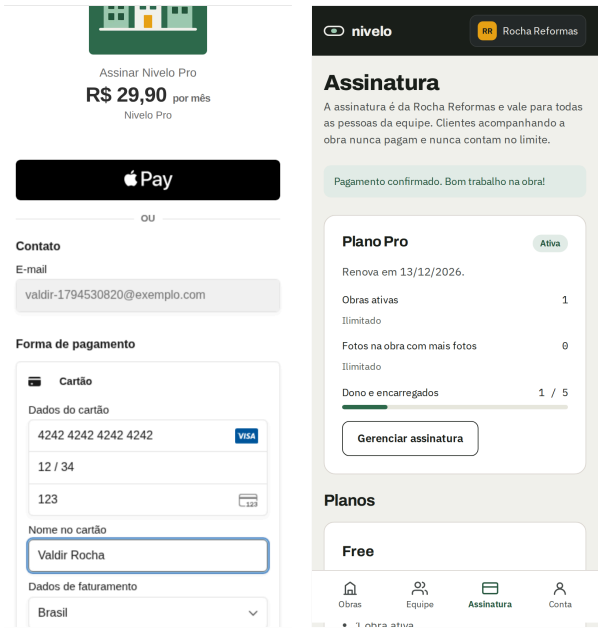


Figura 18: Atividade 11 — checkout do Stripe com o cartão 4242 e o plano Pro depois do webhook

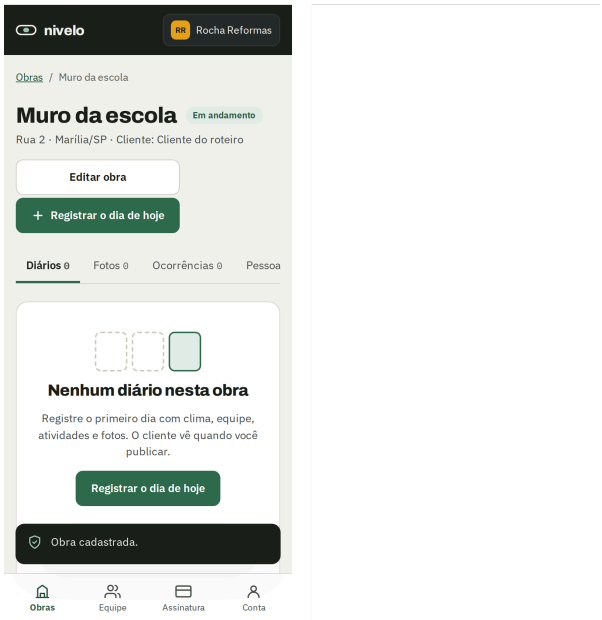


Figura 19: Atividade 11 — a 2ª obra criada no Pro e o portal do cliente



Figura 20: Atividade 11 — painel do admin com assinaturas ativas e MRR de teste

> **Falta para fechar esta ficha (rascunho de 13/11):** o print da volta ao Free depois do cancelamento no portal e do MRR caindo R$ 29,90 (o portal precisa estar com cancelamento imediato); os prints do Stripe em modo de teste (assinatura e Developers ⟶ Webhooks com 200); a issue de entrega com o link do commit; a data e a hora reais acima.

Dificuldades e como foram resolvidas. O checkout em produção respondeu 503 (“Não foi possível falar com o meio de pagamento”). O log passou a registrar o motivo da recusa do Stripe, e ele mostrou `No such price: 'prod_...'`: a variável do preço estava com o ID do produto no lugar do ID do preço (`price_...`). Bastou trocar o valor na stack e recriar o container; o diagnóstico ficou documentado em `docs/STRIPE.md`. Outro ponto foi o webhook receber eventos fora de ordem; em vez de aplicar cada evento como veio, a `api` sempre busca o estado atual da assinatura no Stripe.

6 Considerações finais

O bimestre mostrou que transformar um sistema que funciona num produto que se cobra é, sobretudo, trabalho de fronteiras: entre empresas (tenancy), entre papéis (403 decidido no servidor), entre o que o usuário vê e o que o pagamento confirma (só o webhook muda o plano) e entre o domínio e os fornecedores (a camada anticorrupção do Stripe). Cada uma dessas fronteiras virou teste automatizado e roda no CI a cada push.

O mais difícil foi manter a infraestrutura congelada (mesma stack, mesmo subdomínio, 1 GB por container) enquanto o domínio inteiro era trocado, e diagnosticar problemas de produção sem acesso direto ao painel do Stripe: a lição foi registrar no log o motivo exato de cada falha externa. O que eu faria diferente é começar o roteiro de prints automatizado mais cedo, junto com o código, e não no fim.

Para o Nivelo virar um produto de verdade faltam: o relatório RDO em PDF por obra e período, o clima automático pela Open-Meteo, o resumo semanal por e-mail ao cliente, o link público somente leitura da obra, o refresh token rotativo, o registro offline do diário no celular e o plano anual com desconto.

7 Declaração de autoria

Declaro que este relatório foi elaborado por mim, individualmente, e que as evidências apresentadas correspondem a entregas de minha autoria, verificáveis nos links informados.

Marcio Gabriel Schinor Mazega

Pompeia, 13/11/2026